main.c

Share Run

```c
#include<stdio.h>
#include<stdlib.h>
struct node
{
    int data;
    struct node *next;
    struct node *prev;
};
struct node *head = NULL;

// Insert at beginning
void insert_begin(int val)
{
    struct node *newnode,*temp;

    newnode = (struct node*)malloc(sizeof(struct node));
    newnode->data = val;

    if(head == NULL)
    {
        head = newnode;
        newnode->next = head;
        newnode->prev = head;
    }
    else
    {
        temp = head->prev;
        newnode->next = head;
        newnode->prev = temp;

        temp->next = newnode;
        head->prev = newnode;

        head = newnode;
    }
}

// Insert at end
```

Output

Clear

```
Circular Doubly Linked List:
5 <-> 10 <-> 15 <-> 20 <-> 30 <-> HEAD


=== Code Execution Successful ===
```

main.c

Share Run

```c
// Insert at end
void insert_end(int val)
{
    struct node *newnode,*temp;

    newnode = (struct node*)malloc(sizeof(struct node));
    newnode->data = val;

    if(head == NULL)
    {
        head = newnode;
        newnode->next = head;
        newnode->prev = head;
    }
    else
    {
        temp = head->prev;
        temp->next = newnode;
        newnode->prev = temp;

        newnode->next = head;
        head->prev = newnode;
    }
}

// Insert at given position
void insert_pos(int val,int pos)
{
    struct node *newnode,*temp;
    int i;

    newnode = (struct node*)malloc(sizeof(struct node));
    newnode->data = val;

    temp = head;

    for(i=1;i<pos-1;i++)
        temp = temp->next;
```

Output

Clear

```
Circular Doubly Linked List:
5 <-> 10 <-> 15 <-> 20 <-> 30 <-> HEAD


=== Code Execution Successful ===
```

main.c

Share Run

```c
    newnode->next = temp->next;
    newnode->prev = temp;

    temp->next->prev = newnode;
    temp->next = newnode;
}
// Display
void display()
{
    struct node *temp;

    if(head == NULL)
    {
        printf("List empty\n");
        return;
    }
    temp = head;

    do
    {
        printf("%d <-> ",temp->data);
        temp = temp->next;
    }while(temp != head);

    printf("HEAD\n");
}
int main()
{
    insert_begin(10);
    insert_begin(5);
    insert_end(20);
    insert_end(30);
    insert_pos(15,3);

    printf("Circular Doubly Linked List:\n");
    display();
    return 0;
}
```

Output

Clear

```
Circular Doubly Linked List:
5 <-> 10 <-> 15 <-> 20 <-> 30 <-> HEAD


=== Code Execution Successful ===
```

main.c

Share Run

```c
#include<stdio.h>
#include<stdlib.h>
struct node
{
    int data;
    struct node *next;
    struct node *prev;
};
struct node *head = NULL;

// Delete at beginning
void delete_begin()
{
    struct node *temp;

    if(head == NULL)
    {
        printf("List empty\n");
        return;
    }
    temp = head;

    if(head->next == head)
    {
        head = NULL;
    }
    else
    {
        head = head->next;
        head->prev = temp->prev;
        temp->prev->next = head;
    }

    free(temp);
}
// Delete at end
void delete_end()
{
```

Output

Clear

```
Original List:
10 <-> 20 <-> 30 <-> 40 <-> HEAD
After deleting beginning:
20 <-> 30 <-> 40 <-> HEAD
After deleting end:
20 <-> 30 <-> HEAD
After deleting position 2:
20 <-> HEAD


=== Code Execution Successful ===
```

main.c

Share Run

```c
void delete_end()
{
    struct node *temp;

    if(head == NULL)
    {
        printf("List empty\n");
        return;
    }
    temp = head->prev;

    if(temp == head)
    {
        head = NULL;
    }
    else
    {
        temp->prev->next = head;
        head->prev = temp->prev;
    }

    free(temp);
}
// Delete at given position
void delete_pos(int pos)
{
    struct node *temp;
    int i;

    temp = head;

    for(i = 1; i < pos; i++)
        temp = temp->next;

    temp->prev->next = temp->next;
    temp->next->prev = temp->prev;

    if(temp == head)
```

Output

Clear

```
Original List:
10 <-> 20 <-> 30 <-> 40 <-> HEAD
After deleting beginning:
20 <-> 30 <-> 40 <-> HEAD
After deleting end:
20 <-> 30 <-> HEAD
After deleting position 2:
20 <-> HEAD


=== Code Execution Successful ===
```

main.c

Share Run

```c
    if(temp == head)
        head = temp->next;

    free(temp);
}
// Display
void display()
{
    struct node *temp;

    if(head == NULL)
    {
        printf("List empty\n");
        return;
    }
     temp = head;

    do
    {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    } while(temp != head);

    printf("HEAD\n");
}
int main()
{
    // Manually creating circular doubly linked list
    struct node *n1,*n2,*n3,*n4;

    n1 = (struct node*)malloc(sizeof(struct node));
    n2 = (struct node*)malloc(sizeof(struct node));
    n3 = (struct node*)malloc(sizeof(struct node));
    n4 = (struct node*)malloc(sizeof(struct node));

    head = n1;

    n1->data = 10;
```

Output

Clear

```
Original List:
10 <-> 20 <-> 30 <-> 40 <-> HEAD
After deleting beginning:
20 <-> 30 <-> 40 <-> HEAD
After deleting end:
20 <-> 30 <-> HEAD
After deleting position 2:
20 <-> HEAD


=== Code Execution Successful ===
```

main.c

Share Run

```c
    n3 = (struct node*)malloc(sizeof(struct node));
    n4 = (struct node*)malloc(sizeof(struct node));

    head = n1;

    n1->data = 10;
    n1->next = n2;
    n1->prev = n4;

    n2->data = 20;
    n2->next = n3;
    n2->prev = n1;

    n3->data = 30;
    n3->next = n4;
    n3->prev = n2;

    n4->data = 40;
    n4->next = head;
    n4->prev = n3;

    printf("Original List:\n");
    display();

    delete_begin();
    printf("After deleting beginning:\n");
    display();

    delete_end();
    printf("After deleting end:\n");
    display();

    delete_pos(2);
    printf("After deleting position 2:\n");
    display();

    return 0;
}
```

Output

Clear

```
Original List:
10 <-> 20 <-> 30 <-> 40 <-> HEAD
After deleting beginning:
20 <-> 30 <-> 40 <-> HEAD
After deleting end:
20 <-> 30 <-> HEAD
After deleting position 2:
20 <-> HEAD


=== Code Execution Successful ===
```